INACAP · SEDE PUENTE ALTO

Área Tecnologías de Información y Ciberseguridad

INFORME DE EVALUACIÓN SUMATIVA N° 2

Programación Orientada a Objeto Seguro (TI3V21)

Implementación de Software Orientado a Objetos en Python

■ Negocio Asignado	**Farmacia Cruz del Sur S.A.**

Integrante 1	Santiago Ignacio Prufer Navarro	**RUT:** 22.197.782-3
Integrante 2	Bastian Angelo Flores Diaz	**RUT:** 22.272.121-0
Asignatura	Programación Orientada a Objeto Seguro	**Código:** TI3V21
Sección	114-2B-F1	
Docente	Michael Arjel	
Fecha Entrega	Martes 6 de octubre de 2026	

■ **Repositorio GitHub**	https://github.com/santiprufer18-lab/farmacia_ev2

RESUMEN EJECUTIVO

El presente informe documenta la implementación en Python del modelo de dominio para el sistema POS e integración sanitaria de **Farmacia Cruz del Sur S.A.**, correspondiente a la Evaluación Sumativa N°2 de la asignatura Programación Orientada a Objeto Seguro. Se detalla el diagrama de clases implementado, los principios de POO aplicados (Herencia, Encapsulamiento y Polimorfismo), las relaciones del modelo (Composición y Agregación), las tres excepciones personalizadas de dominio, y la integración con APIs externas oficiales (Banco Central de Chile vía mindicador.cl y Red Nacional de Farmacias MINSAL). Finalmente se incluye la validación técnica y crítica del uso de herramientas de Inteligencia Artificial.

1. Diagrama de Clases y Estructura del Código

1.1. Diagrama de Clases (Evaluación Sumativa N°2)

Durante la implementación en Python, el diagrama original de la EV1 fue corregido y refinado para cumplir estrictamente con los principios de POO Seguro exigidos en la rúbrica. Los principales cambios se detallan en la tabla siguiente:

Aspecto / Elemento	Diseño Inicial (EV1)	Implementación Final (EV2)	Justificación Técnica
Jerarquía de Medicamentos	Flechas de herencia apuntaban hacia Receta en el diagrama inicial.	Se corrigió: los tres subtipos (MedSinReceta, MedRecetaSimple, MedRecetaControlado) heredan directamente de Medicamento con super().__init__().	Los subtipos son medicamentos especializados, no recetas.
Manejo de Fechas	Tipo genérico Date sin validación de formato.	Cadenas str validadas con datetime.strptime() en el setter.	Garantiza formato AAAA-MM-DD sin lanzar excepciones no controladas.
Control de Reglas de Negocio	Retorno de valores booleanos (True/False) ante errores.	Lanzamiento de Excepciones de Dominio personalizadas (MedicamentoVencidoException, RecetaInvalidaException, VentaNoAutorizadaException).	Exigencia de POO Seguro: impide continuar ejecuciones inseguras.
Integración Externa	No contemplada en EV1.	Se añadió APIService con conexión en tiempo real a mindicador.cl (USD) y MINSAL (Farmacias Turno).	Funcionalidad de valor diferenciador real del sistema.

1.2. Estructura de Clases Implementadas (directorio model/)

Clase	Archivo	Rol en el Modelo
Medicamento	medicamento.py	Clase base/padre. Atributos comunes: nombre, precio_base, stock, fecha_vencimiento, es_importado. Método polimórfico validar_venta().
MedSinReceta	med_sin_receta.py	Subtipo Venta Libre. Hereda de Medicamento. Sobrescribe validar_venta(): solo verifica vencimiento.
MedRecetaSimple	med_receta_simple.py	Subtipo Venta con Receta Simple. Exige folio y médico emisor válidos (RecetaInvalidaException).
MedRecetaControlado	med_receta_controlado.py	Subtipo Receta Retenida. Exige receta válida, retenida y autorización de Químico Farmacéutico.
Cliente	cliente.py	Entidad comprador. Valida formato de RUT chileno en setter (8-10 caracteres sin puntos ni guión).
Personal	personal.py	Clase base de trabajadores de la farmacia. Valida RUT en setter (≥8 caracteres).
Vendedor	vendedor.py	Hereda de Personal. Asignado a caja registradora (validación: caja > 0).
QuimicoFarmaceutico	quimico_farmaceutico.py	Hereda de Personal. Número de registro sanitario. Método autorizar_controlado(receta).
Receta	receta.py	Entidad Receta Médica. Atributos: folio, médico, retenida, autorizada_por_quimico.
Venta	venta.py	Transacción principal (Composición de DetalleVenta, Agregación de Cliente y Vendedor).
DetalleVenta	detalle_venta.py	Línea de detalle dentro de la venta. Calcula subtotal. Validación de cantidad > 0.
Excepciones	excepciones.py	Tres excepciones personalizadas de dominio que extienden de Exception.

1.3. Herencia — super().__init__()

Los tres subtipos de medicamentos heredan de la clase base **Medicamento** e invocan explícitamente a **super().__init__()** en su constructor, garantizando la correcta inicialización de los atributos comunes. Cada subtipo *sobrescribe* el método **validar_venta()** con su propia lógica de negocio.

Clase Hija	Clase Padre	Comportamiento Especializado
MedSinReceta	Medicamento	Venta libre, sin receta. Solo verifica vencimiento.
MedRecetaSimple	Medicamento	Exige receta con folio y médico. Lanza RecetaInvalidaException.
MedRecetaControlado	Medicamento	Exige receta retenida + autorización QF. Lanza VentaNoAutorizadaException.
Vendedor	Personal	Agrega atributo caja_asignada con validación (caja > 0).
QuimicoFarmaceutico	Personal	Agrega num_registro y método autorizar_controlado(receta).

Fragmento representativo — **MedRecetaControlado** (model/med_receta_controlado.py):

```
# Archivo: model/med_receta_controlado.py
from model.medicamento import Medicamento
from model.excepciones import VentaNoAutorizadaException, RecetaInvalidaException

class MedRecetaControlado(Medicamento):
    """Representa medicamentos controlados: requieren receta retenida y autorización QF."""
    def __init__(self, nombre: str, precio_base: float, stock: int,
                 fecha_vencimiento: str, es_importado: bool = False,
                 requiere_retencion: bool = True):
        # Invocación obligatoria al constructor de la superclase
        super().__init__(nombre, precio_base, stock, fecha_vencimiento, es_importado)
        self._requiere_retencion = requiere_retencion
```

Fragmento — **MedSinReceta** (model/med_sin_receta.py):

```
class MedSinReceta(Medicamento):
    """Medicamentos de venta libre. No requieren receta médica."""
    def __init__(self, nombre: str, precio_base: float, stock: int,
                 fecha_vencimiento: str, es_importado: bool = False):
        super().__init__(nombre, precio_base, stock, fecha_vencimiento, es_importado)

    def validar_venta(self, receta=None) -> bool:
        super().validar_venta(receta)  # Solo verifica vencimiento
        return True
```

1.4. Agregación — Cliente y Vendedor en Venta

La clase **Venta** recibe objetos previamente creados (**Cliente** y **Vendedor**) mediante inyección de dependencias en el constructor. Si la venta se elimina, los objetos *cliente* y *vendedor* continúan existiendo de manera independiente, lo que constituye una relación de **Agregación**.

```
# Archivo: model/venta.py
class Venta:
    def __init__(self, cliente: Cliente, vendedor: Personal, fecha: str = None):
        self._cliente  = cliente   # AGREGACIÓN: objeto Cliente preexistente
        self._vendedor = vendedor  # AGREGACIÓN: objeto Vendedor preexistente
        self._detalles = []        # COMPOSICIÓN: Venta administra sus DetalleVenta
        self._total    = 0.0
```

1.5. Composición — DetalleVenta dentro de Venta

Los objetos **DetalleVenta** son creados e instanciados *dentro* de la clase **Venta**, en el método **agregar_detalle()**. Un **DetalleVenta** no puede existir fuera del contexto de una Venta específica: su ciclo de vida depende completamente de ella. Esto es **Composición**.

```
# Archivo: model/venta.py
def agregar_detalle(self, medicamento: Medicamento, cantidad: int,
                    receta: Receta = None, valor_dolar: float = 950.0) -> DetalleVenta:
    # 1. Validación polimórfica según la clase concreta del medicamento
    medicamento.validar_venta(receta)
```

```
# 2. Validación de stock suficiente
if medicamento.stock < cantidad:
    raise ValueError(f'Stock insuficiente: {medicamento.stock} disponibles.')
# 3. Descontar stock
medicamento.stock -= cantidad
# 4. COMPOSICIÓN: Venta instancia y administra sus propios DetalleVenta
detalle = DetalleVenta(medicamento, cantidad, receta, valor_dolar)
self._detalles.append(detalle)
self.calcular_total(valor_dolar)
return detalle
```

2. Principios de Programación Orientada a Objetos

2.1. Encapsulamiento con @property y Validación en Setter

Todos los atributos sensibles de las clases son privados (prefijo _) y su acceso se controla mediante **@property** y **setters** con validación. Esto garantiza la integridad de los datos en todo momento.

Clase	Atributo Encapsulado	Validación en Setter
Medicamento	precio_base	Debe ser > 0. Lanza ValueError si se ingresa valor inválido.
Medicamento	stock	No puede ser negativo. Lanza ValueError si stock < 0.
Medicamento	fecha_vencimiento	Valida formato AAAA-MM-DD con datetime.strptime(). Lanza ValueError.
Cliente	rut	Valida longitud: entre 8 y 10 caracteres (sin puntos ni guión).
Personal	rut	Valida longitud mínima: ≥ 8 caracteres.
Vendedor	caja_asignada	Número de caja debe ser > 0. Lanza ValueError.
DetalleVenta	cantidad	Cantidad debe ser ≥ 1. Lanza ValueError si cantidad ≤ 0.

Ejemplo concreto — **precio_base** y **fecha_vencimiento** en Medicamento (model/medicamento.py):

```
# Archivo: model/medicamento.py
@property
def precio_base(self) -> float:
    return self._precio_base

@precio_base.setter
def precio_base(self, valor: float):
    if valor <= 0:
        raise ValueError("El precio base del medicamento debe ser mayor a 0.")
    self._precio_base = float(valor)

@fecha_vencimiento.setter
def fecha_vencimiento(self, fecha_str: str):
    try:
        datetime.strptime(fecha_str, "%Y-%m-%d")
        self._fecha_vencimiento = fecha_str
    except ValueError:
        raise ValueError("La fecha de vencimiento debe tener el formato AAAA-MM-DD.")
```

Ejemplo concreto — validación de **RUT de cliente** (model/cliente.py):

```
# Archivo: model/cliente.py
@property
def rut(self) -> str:
    return self._rut

@rut.setter
def rut(self, valor: str):
    val_limpio = str(valor).replace('.', '').replace('-', '').strip()
    if len(val_limpio) < 8 or len(val_limpio) > 10:
        raise ValueError(f"RUT '{valor}' no es válido (debe tener entre 8 y 9 dígitos más DV).")
    self._rut = valor
```

2.2. Polimorfismo — validar_venta() en los Tres Subtipos

Cada tipo de medicamento redefine el comportamiento de **validar_venta()** según las reglas farmacéuticas y legales que aplican a su categoría. El código cliente (clase **Venta**) invoca medicamento.validar_venta(receta) de forma polimórfica, **sin if isinstance(...)**, respetando el principio Open/Closed (OCP).

Clase	Método Polimórfico	Comportamiento Específico	Excepción Lanzada

MedSinReceta	validar_venta(receta=None)	Llama a super().validar_venta(). Solo bloquea si el medicamento está vencido.	MedicamentoVencidoException
MedRecetaSimple	validar_venta(receta=None)	Verifica vencimiento y luego exige que receta no sea None y tenga folio + médico válidos.	RecetaInvalidaException
MedRecetaControlado	validar_venta(receta=None)	Verifica vencimiento, exige receta válida, que esté retenida y autorizada por QF.	RecetaInvalidaException / VentaNoAutorizadaException

Fragmento — polimorfismo en **MedRecetaControlado** (model/med_receta_controlado.py):

```
def validar_venta(self, receta=None) -> bool:
    """Sobrescribe validar_venta(): exige receta válida, retenida y autorizada por QF."""
    super().validar_venta(receta)  # Verifica vencimiento base
    if receta is None or not receta.es_valida():
        raise RecetaInvalidaException(
            f"Venta rechazada: '{self.nombre}' requiere receta médica válida.")
    if self._requiere_retencion and not receta.retenida:
        raise VentaNoAutorizadaException(
            f"Venta rechazada: '{self.nombre}' requiere retención de receta.")
    if not receta.autorizada_por_quimico:
        raise VentaNoAutorizadaException(
            f"Venta rechazada: '{self.nombre}' requiere autorización de QF.")
    return True
```

3. Evidencia de Funcionamiento y Manejo de Excepciones

3.1. Excepciones Personalizadas del Dominio (model/excepciones.py)

Se implementaron **tres excepciones de dominio** que extienden directamente de **Exception**, siguiendo el principio de diseño que establece que cada regla de negocio que impide una operación debe contar con su propia excepción semántica.

Excepción	Descripción	Clase/Método que la Lanza	Regla de Negocio que Protege
MedicamentoVencidoException	Se lanza cuando se intenta vender un medicamento cuya fecha de vencimiento ya pasó.	Medicamento.validar_venta() → esta_vencido()	Regla 1: No vender medicamentos vencidos.
RecetaInvalidaException	Se lanza cuando la receta es None, o no tiene folio/médico válidos para un medicamento que la exige.	MedRecetaSimple.validar_venta() MedRecetaControlado.validar_venta()	Regla 2: Exigir receta válida para medicamentos controlados y de receta simple.
VentaNoAutorizadaException	Se lanza cuando un medicamento controlado no tiene receta retenida o no fue autorizado por el Químico Farmacéutico.	MedRecetaControlado.validar_venta()	Regla 3: Controlados requieren QF.

Implementación completa (model/excepciones.py):

```
# Archivo: model/excepciones.py
"""Módulo de Excepciones Personalizadas del Dominio de Farmacia."""

class MedicamentoVencidoException(Exception):
    """Excepción lanzada cuando se intenta vender un medicamento vencido."""
    def __init__(self, nombre_medicamento: str, fecha_vencimiento: str):
        self.nombre_medicamento = nombre_medicamento
        self.fecha_vencimiento  = fecha_vencimiento
        super().__init__(
            f"No se puede vender '{nombre_medicamento}': vencido desde {fecha_vencimiento}.")

class RecetaInvalidaException(Exception):
    """Excepción cuando la receta médica no es válida o falta para el medicamento."""
    def __init__(self, mensaje: str = 'La receta médica no es válida o no existe.'):
        super().__init__(mensaje)

class VentaNoAutorizadaException(Exception):
    """Excepción cuando la venta no cuenta con la autorización de un QF."""
    def __init__(self, mensaje: str = 'Venta rechazada: requiere autorización de QF.'):
        super().__init__(mensaje)
```

3.2. Captura Controlada en main.py — Terminal POS

En el script principal **main.py**, todas las operaciones de venta están protegidas con bloques **try/except**. Las reglas de negocio se provocan a propósito al intentar ventas inválidas, el programa captura la excepción, informa al usuario y continúa ejecutándose sin caerse.

```
# Archivo: main.py — Fragmento del Terminal POS (menu_pos_venta)
try:
    venta.agregar_detalle(
        medicamento=med_sel, cantidad=cant,
        receta=receta_asociada, valor_dolar=valor_dolar
    )
    print(f' ✔■ {cant} unidad(es) de "{med_sel.nombre}" agregadas.')

except (MedicamentoVencidoException, RecetaInvalidaException,
        VentaNoAutorizadaException) as e_negocio:
    print(f'\n ■■ [OPERACIÓN DENEGADA POR SEGURIDAD]: {e_negocio}')

except ValueError as e_val:
    print(f'\n ■ [ENTRADA INVÁLIDA]: {e_val}')
```

3.3. Casos de Prueba — Resumen de Ejecución de main.py

El script **main.py** implementa un sistema completo de Terminal POS interactivo. A continuación se detallan los casos de prueba que demuestran el correcto funcionamiento del polimorfismo y la captura de excepciones:

N°	Tipo de Prueba	Acción Ejecutada	Resultado Esperado y Observado
1	Venta Libre (MedSinReceta)	Agregar Paracetamol 500mg (stock: 120) sin receta.	✔■ Venta exitosa. Stock descontado. Boleta generada con IVA (19%).
2	Receta Simple (MedRecetaSimple)	Agregar Amoxicilina 500mg con folio de receta y médico emisor. Medicamento importado: precio en USD convertido a CLP en tiempo real.	✔■ Venta autorizada. Precio calculado con tipo de cambio oficial (mindicador.cl).
3	Regla 1 — Medicamento Vencido	Intentar vender 'Aspirina 100mg (VENCIDA TEST)' (fecha: 2023-01-01, actualmente vencida).	■■ MedicamentoVencidoException capturada. Programa informa el error y continúa.
4	Regla 2 — Controlado sin QF	Intentar vender Clonazepam 2mg sin autorización del Químico Farmacéutico (receta no autorizada).	■■ VentaNoAutorizadaException capturada. Programa informa el error y continúa.
5	Regla 3 — Receta no Retenida	Intentar vender medicamento controlado con receta presentada pero sin retener físicamente.	■■ VentaNoAutorizadaException capturada. Programa bloquea la venta y continúa.
6	Setter — RUT Inválido	Ingresar RUT de cliente '123' (menos de 8 caracteres).	■ ValueError capturada desde el setter. Programa informa el error y continúa.
7	Autorización QF — Receta Controlada	Químico Farmacéutico 'Dra. María González' (Reg. QF-88421) autoriza receta y retiene. Luego venta de Tramadol 50mg.	✔■ Receta autorizada. Venta de controlado aprobada y registrada.
8	Historial de Ventas y Boleta	Reemisión de boleta electrónica con desglose IVA 19% y timbre SII desde VentaService.	✔■ Boleta emitida con NETO, IVA y TOTAL.
9	Consulta MINSAL	Consultar farmacias de turno filtrando por comuna.	✔■ Lista de farmacias obtenida desde API MINSAL (o datos de respaldo offline).
10	Auditoría del Sistema	Ver bitácora de eventos: SISTEMA_INIT, DOLAR_SYNC_OK, VENTA_REGISTRADA, RECETA_AUTORIZADA.	✔■ Registro completo de trazabilidad de seguridad.

4. Validación Técnica y Uso Crítico de Inteligencia Artificial

Durante el desarrollo se utilizaron asistentes de Inteligencia Artificial (Claude/Gemini) para acelerar la estructuración de la lógica de servicios y el código repetitivo (boilerplate). Todo el código generado fue evaluado técnicamente por el equipo antes de su adopción, modificación o descarte, siguiendo los criterios de la rúbrica (criterio 2.1.5).

4.1. Componentes Adoptados desde IA

Componente	Decisión	Justificación Técnica
Integración API — APIService	Adoptada	Se adoptó la estructura propuesta por la IA para consumir las APIs de mindicador.cl (cotización oficial USD) y MINSAL (farmacias de turno) usando urllib.request nativo de Python. Se validó que no requiere dependencias externas, lo que cumple con el criterio de cero dependencias de terceros.
Patrón AuditoriaService	Adoptado	Se adoptó el patrón de registro de eventos centralizado con método de clase @classmethod y lista estática _log_eventos. Garantiza trazabilidad de seguridad para todos los eventos críticos (ventas, autorizaciones, sincronizaciones).
Estructura de VentaService	Adoptada con ajustes	Se adoptó la separación de responsabilidades en VentaService (registrar_venta y generar_boleta_texto). Se ajustó la boleta para incluir desglose IVA 19% y referencia SII, cumpliendo la normativa tributaria chilena.

4.2. Ejemplos Concretos de Modificación y Descarte

Propuesta Inicial de la IA	Decisión del Equipo	Justificación Técnica / Razón POO
La IA propuso usar condicionales if isinstance(med, MedRecetaControlado): dentro del bucle de venta en main.py para validar la receta.	DESCARTADO MODIFICADO	Se descartó por violar el principio Open/Closed (OCP) y romper el polimorfismo exigido en la rúbrica. La responsabilidad se trasladó al método polimórfico validar_venta(receta) sobrescrito en cada subclase. Esto permite agregar nuevos tipos de medicamentos sin modificar el código cliente.
La IA sugirió retornar False o imprimir mensajes con print() al detectar una venta inválida.	DESCARTADO REEMPLAZADO	Se reemplazó por el lanzamiento explícito de excepciones de dominio (MedicamentoVencidoException, VentaNoAutorizadaException). Esto garantiza que ninguna parte del sistema pueda ignorar silenciosamente un error, cumpliendo el criterio de robustez y seguridad del software.
La IA propuso usar librerías externas (requests) para el consumo de APIs REST.	DESCARTADO	Se descartó por política de cero dependencias externas. Se reemplazó por urllib.request (biblioteca estándar de Python), garantizando portabilidad y que el sistema funcione en cualquier entorno sin instalación adicional.
La IA propuso usar variables de instancia directas (sin _ y sin property/setter).	DESCARTADO	Se descartó porque viola el principio de Encapsulamiento exigido en la rúbrica. Todos los atributos sensibles fueron encapsulados con prefijo _, @property y setter con validación, impidiendo la asignación de valores inválidos desde fuera.

5. Servicios Adicionales Implementados

Además del modelo de clases core, se implementaron cuatro servicios en el directorio **services/** que dotan al sistema de capacidades de integración, operación y auditoría:

Servicio	Archivo	Funcionalidades
APIService	api_service.py	obtener_dolar_tiempo_real(): Consulta cotización oficial USD desde mindicador.cl con fallback a $950 CLP. obtener_farmacias_turno(comuna): Consulta Red Nacional MINSAL con filtro por comuna y fallback offline.
InventarioService	inventario_service.py	Gestiona el catálogo con 11 medicamentos (4 libres, 4 receta simple, 3 controlados incluyendo 1 importado en USD). Métodos: listar_todos(), buscar_por_nombre(), obtener_vencidos(), agregar_medicamento().
VentaService	venta_service.py	registrar_venta(venta): Persiste la venta y registra evento de auditoría. generar_boleta_texto(venta, valor_dolar): Genera boleta electrónica con desglose IVA 19% y referencia SII.
AuditoriaService	auditoria_service.py	Registro centralizado de eventos de seguridad: SISTEMA_INIT, DOLAR_SYNC_OK, DOLAR_SYNC_OFFLINE, MINSAL_SYNC_OK, VENTA_REGISTRADA, RECETA_AUTORIZADA. Patrón Singleton vía @classmethod con lista estática.

5.1. Módulo de Autorización de Recetas Controladas (QuimicoFarmaceutico)

El sistema implementa un flujo completo de autorización para medicamentos controlados. El **QuimicoFarmaceutico** es el único rol facultado para autorizar recetas retenidas:

```
# Archivo: model/quimico_farmaceutico.py
class QuimicoFarmaceutico(Personal):
    """Profesional farmacéutico facultado para autorizar medicamentos controlados."""
    def __init__(self, rut: str, nombre: str, num_registro: str):
        super().__init__(rut, nombre)
        self._num_registro = num_registro

    def autorizar_controlado(self, receta: Receta) -> bool:
        """Autoriza una receta retenida para venta de medicamentos controlados."""
        if receta and receta.es_valida() and receta.retenida:
            receta.autorizada_por_quimico = True
            return True
        return False
```

6. Conclusión

La solución desarrollada implementa rigurosamente todos los principios de **Programación Orientada a Objetos Seguro** exigidos en la rúbrica:

Criterio Cumplido	Evidencia en el Código
✔■ Herencia con super().__init__()	Cinco cadenas de herencia: MedSinReceta, MedRecetaSimple y MedRecetaControlado heredan de Medicamento. Vendedor y QuimicoFarmaceutico heredan de Personal.
✔■ Encapsulamiento con @property	Siete atributos críticos encapsulados con property + setter y validación activa: precio_base, stock, fecha_vencimiento (Medicamento), rut (Cliente/Personal), caja_asignada (Vendedor), cantidad (DetalleVenta).
✔■ Polimorfismo puro	El método validar_venta() es sobrescrito en los tres subtipos de medicamentos, sin condicionales isinstance() en el código cliente, respetando el principio Open/Closed.
✔■ Composición y Agregación	Venta instancia y posee sus DetalleVenta (Composición). Venta recibe Cliente y Vendedor preexistentes (Agregación), cuyo ciclo de vida es independiente.
✔■ Tres Excepciones de Dominio	MedicamentoVencidoException, RecetaInvalidaException y VentaNoAutorizadaException, cada una representando una regla de negocio específica que impide una operación.
✔■ Integración con APIs Externas	Cotización oficial USD desde mindicador.cl (Banco Central de Chile) y Red Nacional de Farmacias de Turno desde MINSAL, con manejo de errores de conexión y valor de respaldo.
✔■ Trazabilidad y Auditoría	AuditoriaService registra todos los eventos críticos del sistema con timestamp, tipo, usuario y detalle, garantizando la trazabilidad de seguridad.
✔■ Cero Dependencias Externas	El sistema funciona con bibliotecas estándar de Python (urllib.request, datetime, json, sys), sin requerir pip install de terceros.

El modelo de dominio es **robusto, encapsulado y extensible**, respetando las leyes farmacéuticas y tributarias de Chile. El sistema puede ser extendido (nuevos tipos de medicamentos, nuevos servicios de auditoría) sin modificar el código existente, cumpliendo así el principio Open/Closed de SOLID.